Lab Activity 3.3: Construct form validation using Bootstrap and JavaScript

STUDENT INFORMATION

Name	Registration No	Date
NUR ALIA ASYIQIN BINTI KHAIRUL ANUAR	10DIT24F1093	11/3/2026

Learning Outcomes:

a) Construct client-side form validation using JavaScript.
b) Construct Bootstrap form components for improved user experience.
c) Perform form submission and validation errors gracefully.

Step 1: Setting Up the Document Structure & Styling

Create a new file named `passport_form.html`. We will start by defining the **Purple Theme** using CSS variables and styling the passport photo container to meet the **35mm x 45mm** requirement.

HTML

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Lab 3.3: Passport Application Form</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap
.min.css" rel="stylesheet">
<style>
    :root { --primary-purple: #6f42c1; --hover-purple: #59359a; --
light-purple: #f3e5f5; }
    body { background-color: var(--light-purple); padding: 40px 20px;
}
    .form-container { max-width: 800px; margin: auto; background:
white; padding: 40px; border-radius: 15px; border-top: 8px solid
var(--primary-purple); }
    .text-purple { color: var(--primary-purple); }
    .bg-purple { background-color: var(--primary-purple) !important;
}
    .btn-purple { background-color: var(--primary-purple); color:
white; }
```

```
    .btn-purple:hover { background-color: var(--hover-purple); color:
white; }
    .passport-photo { width: 3.5cm; height: 4.5cm; object-fit: cover;
border: 3px solid #f8f9fa; outline: 1px solid #dee2e6; display:
block; margin: 0 auto 10px auto; }
    .form-control:focus, .form-select:focus { border-color: var(--
primary-purple); box-shadow: 0 0 0 0.25rem rgba(111, 66, 193, 0.25);
}
    #rangeValue { background-color: var(--primary-purple); color:
white; padding: 2px 10px; border-radius: 10px; font-size: 0.9rem; }
    #successMessage { display: none; }
</style>
</head>
```

Step 2: Constructing the UI (Form Components)

Add the HTML body. This includes the **Success Alert**, **Passport Image**, **Floating Labels**, and **Input Groups**. Note the use of `novalidate` in the form tag to allow custom JavaScript handling.

HTML

```
<div class="container">
    <div class="form-container shadow-lg">

        <div class="mb-3">
            <a href="LAB2_1.html" class="text-decoration-none text-
purple small fw-bold">&larr; Back to Profile</a>
        </div>

        <h2 class="mb-4 text-center fw-bold text-purple">Passport
Application</h2>
        <hr class="mb-5">

        <div id="successMessage" class="alert alert-success alert-
dismissible fade show" role="alert">
            <strong>Success!</strong> Your application has been
validated and submitted gracefully.
            <button type="button" class="btn-close" data-bs-
dismiss="alert" aria-label="Close"></button>
        </div>

        <form class="row g-4 needs-validation" id="passportForm"
novalidate>

            <div class="col-12 text-center">
                <img src="img/profile.png" class="passport-photo
rounded shadow-sm" alt="Profile">
                <p class="text-muted small">Standard Passport Size
(35mm x 45mm)</p>
            </div>

            <div class="col-md-6">
                <div class="form-floating">
```

```
                    <input type="text" class="form-control"
id="firstName" placeholder="First Name" required>
                    <label for="firstName">First Name</label>
                    <div class="invalid-feedback">First name is
required.</div>
                </div>
            </div>

            <div class="col-md-6">
                <div class="form-floating">
                    <input type="text" class="form-control"
id="lastName" placeholder="Last Name" required>
                    <label for="lastName">Last Name</label>
                    <div class="invalid-feedback">Last name is
required.</div>
                </div>
            </div>

            <div class="col-md-12">
                <label for="userEmail" class="form-label fw-
bold">Contact Email</label>
                <div class="input-group has-validation">
                    <span class="input-group-text bg-purple text-
white">@</span>
                    <input type="email" class="form-control"
id="userEmail" placeholder="e.g. name@mail.com" required>
                    <div class="invalid-feedback">Please enter a
valid email address.</div>
                </div>
            </div>

            <div class="col-md-6">
                <label for="citizenStatus" class="form-label fw-
bold">Citizenship</label>
                <select class="form-select" id="citizenStatus"
required>
                    <option selected disabled value="">Select
status...</option>
                    <option>Citizen</option>
                    <option>Permanent Resident</option>
                </select>
                <div class="invalid-feedback">Selection
required.</div>
            </div>

            <div class="col-md-6">
                <label for="yearsResidency" class="form-label fw-
bold">
                    Residency Years: <span id="rangeValue">25</span>
                </label>
                <input type="range" class="form-range" min="1"
max="50" value="25" id="yearsResidency">
            </div>

            <div class="col-md-12">
                <label class="form-label d-block fw-
bold">Gender</label>
                <div class="form-check form-check-inline">
```

```
                    <input class="form-check-input" type="radio"
name="gender" id="male" required>
                    <label class="form-check-label"
for="male">Male</label>
                </div>
                <div class="form-check form-check-inline">
                    <input class="form-check-input" type="radio"
name="gender" id="female" required>
                    <label class="form-check-label"
for="female">Female</label>
                </div>
                <div class="invalid-feedback">Please select a
gender.</div>
            </div>

            <div class="col-12">
                <div class="form-check">
                  <input class="form-check-input" type="checkbox"
value="" id="invalidCheck" required>
                  <label class="form-check-label" for="invalidCheck">
                    I confirm that all information provided is
accurate.
                  </label>
                  <div class="invalid-feedback">You must agree before
submitting.</div>
                </div>
            </div>

            <div class="col-md-6">
                <div class="form-floating">
                    <input type="password" class="form-control"
id="password" placeholder="Password" required>
                    <label for="password">Password</label>
                    <div class="invalid-feedback">Please enter a
password.</div>
                </div>
            </div>

            <div class="col-md-6">
                <div class="form-floating">
                    <input type="password" class="form-control"
id="confirmPassword" placeholder="Confirm Password" required>
                    <label for="confirmPassword">Confirm
Password</label>
                    <div class="invalid-feedback"
id="confirmFeedback">Passwords do not match!</div>
                </div>
            </div>

            <div class="col-12 mt-5 d-flex gap-2">
                <button class="btn btn-purple w-75 py-3 fw-bold
shadow-sm" type="submit">SUBMIT APPLICATION</button>
                <button class="btn btn-outline-secondary w-25 py-3
fw-bold shadow-sm" type="reset" id="resetBtn">RESET</button>
            </div>

        </form>
    </div>
```

```
</div>
```

Step 3: JavaScript Logic (Validation & Interaction)

Implement the script to handle **Range Slider** updates, **Password Matching**, and **Graceful Error Handling**.

HTML

```
<script>
    (function () {
        'use strict'

        const form = document.getElementById('passportForm');
        const successDiv = document.getElementById('successMessage');
        const resetBtn = document.getElementById('resetBtn');
        const rangeInput = document.getElementById('yearsResidency');
        const rangeValue = document.getElementById('rangeValue');

        // Update range value display
        rangeInput.addEventListener('input', function() {
            rangeValue.textContent = this.value;
        });

        // Form Validation Logic
        form.addEventListener('submit', function (event) {
            if (!form.checkValidity()) {
                event.preventDefault();
                event.stopPropagation();
                successDiv.style.display = 'none';
            } else {
                event.preventDefault();
                successDiv.style.display = 'block';
                window.scrollTo({ top: 0, behavior: 'smooth' });
            }
            form.classList.add('was-validated');
        }, false);

        // Reset Logic
        resetBtn.addEventListener('click', function() {
            form.classList.remove('was-validated');
            successDiv.style.display = 'none';
            rangeValue.textContent = "25"; // Reset range display to
default
        });

    })();
    (function () {
    'use strict'

    const form = document.getElementById('passportForm');
    const password = document.getElementById('password');
    const confirmPassword =
document.getElementById('confirmPassword');

    // Fungsi untuk menyemak padanan kata laluan
```

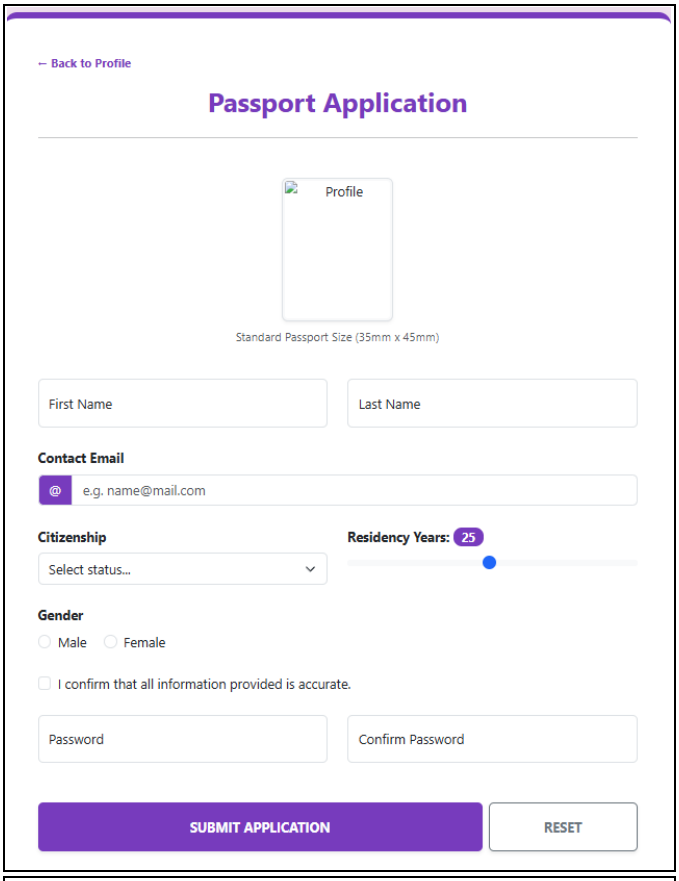
```
    function validatePassword() {
        if (password.value !== confirmPassword.value) {
            // Jika tidak sama, set ralat custom
            confirmPassword.setCustomValidity("Passwords do not
match");
        } else {
            // Jika sama, kosongkan ralat (sah)
            confirmPassword.setCustomValidity("");
        }
    }

    // Jalankan fungsi setiap kali pengguna menaip
    password.addEventListener('change', validatePassword);
    confirmPassword.addEventListener('keyup', validatePassword);

    // Kekalkan logik submit anda yang sedia ada
    form.addEventListener('submit', function (event) {
        // validatePassword dipanggil sekali lagi sebelum submit
untuk kepastian
        validatePassword();

        if (!form.checkValidity()) {
            event.preventDefault();
            event.stopPropagation();
        } else {
            event.preventDefault();
            document.getElementById('successMessage').style.display =
'block';
            window.scrollTo({ top: 0, behavior: 'smooth' });
        }
        form.classList.add('was-validated');
    }, false);

    // ... (Logik Range dan Reset anda yang lain)
})();
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.b
undle.min.js"></script>
```

Task Recap for Students:

1. **Step 1:** Copy CSS to define the look and the specific **3.5cm x 4.5cm** image size.
2. **Step 2:** Construct the form using Bootstrap's grid and specialized components.
3. **Step 3:** Apply the JavaScript to enable live slider updates and ensure the passwords match before submission.
4. **Observation:** Notice how the `was-validated` class triggers red/green borders and messages gracefully without refreshing the page.

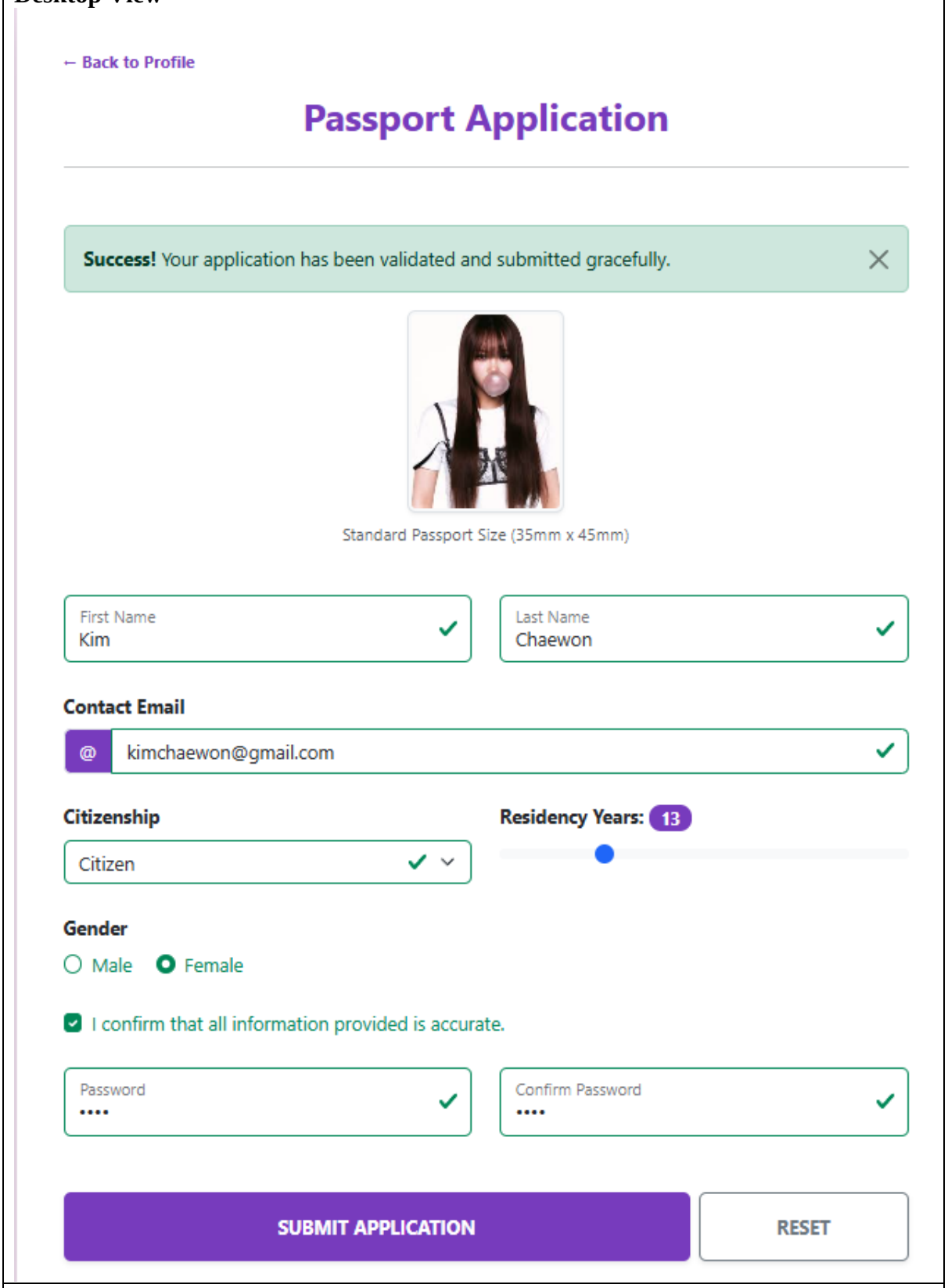
STEP-BY-STEP VALIDATION:

Please provide a **screenshot** for every completed step. For each image, write a short **observation** explaining the logic behind the code you just implemented. This helps solidify your mastery of Bootstrap!

Progress Step	**Screenshot of Output**	**Your Observation & Understanding**
Step 1		
Step 1		
Cont…		

5. SCREENSHOT OUTPUT (COMPLETE):

Desktop View



Mobile View

6. LAB SUBMISSION TASK

Ensure you have completed all steps successfully!

1. **Compress** your folder: `RegNo_Date.zip`.
2. **Report:** Create a PDF containing all screenshots and explanations: `Report_RegNo_Date.pdf`.

Happy Coding! Success is in your hands!